

Autonomous Drone Navigation Using Deep Reinforcement Learning

COMP9444 Project Team

Abstract—This project develops a visual deep reinforcement learning system for autonomous multirotor navigation in Microsoft AirSim. The drone observes an 84 x 84 front depth image together with relative goal position and velocity, and chooses among six discrete body-frame actions without an obstacle map. We implement a vanilla Deep Q-Network (DQN) baseline, a stabilised Proximal Policy Optimisation (PPO) agent trained directly on the final route, and PPO trained through a three-stage distance curriculum. All methods consume 45,000 environment interactions and are selected by validation before an independent 50-episode test. Deterministic PPO trained from scratch achieves 98% success with 2% collision, compared with 16%/72% for DQN and 68%/2% for curriculum PPO. The results show that stable on-policy optimisation can learn a repeatable depth-driven detour, while the curriculum reduces collisions but introduces a 30% timeout rate. The evidence is limited to one route and one training seed, so cross-route generalisation remains future work.

Keywords—AirSim, autonomous drone navigation, deep reinforcement learning, DQN, PPO, curriculum learning.

I. INTRODUCTION

Autonomous flight requires a controller to convert high-dimensional visual observations into safe actions under uncertain geometry. Hand-engineered planners usually depend on maps, calibrated sensing, or explicit obstacle models; these assumptions are difficult to maintain in cluttered and changing environments. Deep reinforcement learning (DRL) offers an alternative in which perception and action selection are learned jointly from simulator interaction.

We study point-to-point navigation in the AirSimNH neighbourhood scene. The agent must travel approximately 32.55 m, avoid houses and vegetation, remain within a safe altitude band, and stop within a 2 m goal radius. The central question is whether PPO learns this visual route more reliably than a vanilla DQN baseline, and whether a progressive target-distance curriculum improves PPO under the same interaction budget.

This question is separated into four measurable objectives: (1) learn a policy from online visual interaction rather than stored obstacle coordinates; (2) complete the route with few collisions or altitude violations; (3) compare algorithms under an equal 45,000-step interaction budget; and (4) determine whether the learned action distribution is robust enough for deterministic deployment. Success rate is therefore the primary measure, while collision, timeout, return, path efficiency, final distance, and perceived clearance explain why a policy succeeds or fails.

Our work is the AirSim-to-Gymnasium environment, visual/state preprocessing, common CNN policy interface, reward and safety design, stable PPO modifications, three-stage transfer procedure, per-step diagnostics, checkpoint selection, and controlled evaluation. AirSim, DQN, PPO, and the curriculum principle originate from prior work [1]–[4]; the implementation and experiments reported here are our own.

II. RELATED WORK

AirSim provides an Unreal Engine-based visual and physical simulator for autonomous vehicles, including multirotor dynamics, cameras, collision queries, and programmatic control [1], [5]. Its repeatability and realistic sensing make it suitable for training without risking physical hardware, although simulator performance alone does not establish real-world transfer.

DQN demonstrated that convolutional networks can learn value functions directly from image observations in discrete-action tasks [2]. Its replay buffer and target network improve stability, but off-policy value learning can be sensitive to sparse terminal rewards and correlated failure modes. PPO instead updates a stochastic actor-critic using a clipped surrogate objective, permitting multiple minibatch epochs while limiting destructive policy changes [3]. Curriculum learning orders training tasks from easier to harder [4]; here, target distance is increased across stages. Unlike these foundational benchmarks, our comparison holds the visual encoder, action space, reward, final route, and total interactions constant.

Prior work supplies the algorithmic principles, not a ready-made solution to this route. DQN was originally demonstrated on stacked game frames, whereas our state combines one metric depth frame with explicit target and velocity features. PPO is normally presented as a general optimisation procedure rather than an AirSim navigation policy. Consequently, the main engineering and research decisions

in this project are the observation fusion, safety-aware reward, simulator reset handling, curriculum transfer, and evaluation protocol.

III. RL TASK AND METHODS

A. Task Formulation and Challenges

The environment is treated as a finite-horizon partially observable Markov decision process. At step t the observation $\mathbf{o}_t = (\mathbf{I}_t, \mathbf{g}_t, \mathbf{v}_t)$ contains a depth image $\mathbf{I}_t$, relative goal vector $\mathbf{g}_t$, and linear velocity $\mathbf{v}_t$; the agent selects one of six actions and receives reward r_t . The transition distribution is produced by AirSim physics and collision geometry. A discount factor of 0.99 values immediate safety while retaining the long-horizon benefit of reaching the goal. Because experience is generated online, conventional dataset properties such as sample count, classes, and train/test labels do not apply. The effective data are temporally correlated transitions collected by the current behaviour policy. The task is challenging because a single forward frame cannot reveal occluded geometry or motion history, progress rewards can favour locally direct motion toward an obstructed goal, and a collision may end an episode before delayed avoidance behaviour receives useful credit. The NED coordinate convention also means that more negative z values are higher, so altitude constraints and vertical actions require careful sign handling.

B. Observation, Actions, and Reward

There is no offline dataset or class label. AirSim generates one transition per interaction. The visual input is a front DepthPerspective frame clipped to 40 m, converted to obstacle intensity 1-depth/40, resized to 84 x 84, and stored as a single normalised channel. A six-dimensional state appends scaled relative goal position ($dx/50, dy/50, dz/10$) and velocity ($vx/10, vy/10, vz/10$). The policy receives neither a map nor obstacle coordinates; collision state is used only for reward, termination, and logging.

Inverse-depth conversion makes nearby surfaces bright and numerically prominent while compressing empty long-range space. Resizing reduces the simulator frame to 7,056 scalar pixels, small enough for repeated CNN updates, and the separate low-dimensional state prevents the network from having to infer the target direction or speed from vision. The trade-off is information loss: thin branches, distant obstacles, and side obstacles outside the front camera can be missed. The observation is therefore suitable for reactive navigation but is not a complete world model.

The discrete actions are forward, left, right, up, down, and hover. Each command lasts 0.35 s at 2 m/s horizontally or 1 m/s vertically. The shaped reward is $r = -0.05 + 2(d_{\text{prev}} - d) - 0.25|z - z_{\text{target}}|$ plus a boundary-margin term. Reaching the goal adds +100; collision or altitude violation adds -100; a 150-step timeout adds -25. Episodes terminate on success, a new collision, altitude outside $-10 \leq z \leq -1$ m in NED coordinates, or timeout.

$$r_t = -0.05 + 2(d_{(t-1)} - d_t) - 0.25|z_t - z_g| - p_{\text{margin}} + r_{\text{terminal}}$$

Here p_{margin} grows linearly when the drone is within 1.5 m of either altitude boundary. The progress term supplies dense feedback, the time cost discourages hovering and unnecessarily long paths, and the terminal terms make safety violations much more expensive than one favourable movement step. Reward components, position, depth, selected action probability, and terminal cause are logged for every

evaluation step, allowing a poor return to be traced to collision, altitude control, timeout, or inefficient motion.

C. Shared Visual Encoder and DQN

All agents use three ReLU convolutional layers: 32 filters (8 x 8, stride 4), 64 filters (4 x 4, stride 2), and 64 filters (3 x 3, stride 1). The resulting 3,136 visual features are concatenated with the six navigation features. DQN maps the 3,142-vector through a 512-unit ReLU layer to six Q-values. It uses $\gamma=0.99$, Adam at $1e-4$, batch size 32, a 50,000-transition uniform replay buffer, Huber TD loss, gradient clipping, a target update every 1,000 interactions, and epsilon decay from 1.0 to 0.05 over 50,000 steps. It intentionally omits Double, Dueling, prioritised replay, and n-step extensions, so it is a transparent baseline rather than an optimised DQN.

$$L_{DQN} = E[\text{Huber}(Q_{\theta}(s_t, a_t) - (r_t + \gamma \max_a Q_{\theta}(s_{t+1}, a))))]$$

Learning starts after 1,000 transitions. One uniformly sampled minibatch is used per environment interaction, and the target network is copied from the policy network every 1,000 interactions. At the end of a 45,000-step run, linear exploration is still approximately $\epsilon=0.145$ because the decay horizon is 50,000 steps; evaluation disables exploration and takes $\arg\max Q$. This design gives DQN broad state coverage but also fills much of the replay buffer with early collision-heavy behaviour.

D. Stable PPO and Curriculum

PPO feeds the shared representation through a 512-unit layer, per-sample layer normalisation, and Tanh. Separate linear actor and critic heads output six logits and one state value. Orthogonal initialisation, reward scaling by 0.1, Huber value loss, advantage normalisation, gradient clipping, and a linear entropy schedule from 0.01 to 0.001 address the activation saturation observed in an earlier implementation. Training uses $\gamma=0.99$, GAE $\lambda=0.95$, clip coefficient 0.2, 500-step rollouts, batch size 64, four epochs, and Adam at $1e-4$.

$$L_{clip} = E[\min(\text{ratio}_t A_t, \text{clip}(\text{ratio}_t, 1-\epsilon, 1+\epsilon) A_t)]$$

Generalised Advantage Estimation trades variance against bias through $\lambda=0.95$. For each rollout, scaled rewards are bootstrapped with critic values, advantages are normalised, and four shuffled minibatch epochs update the actor and critic. The clipped probability ratio constrains sudden policy changes, while entropy encourages early action diversity and decays so that late training can form a sharper deployment policy. Layer normalisation is applied before Tanh so large fused CNN/state activations do not drive all 512 hidden units into saturated regions with near-zero gradients.

Scratch PPO receives all 45,000 interactions on the final 33 m task. Curriculum PPO uses 5,000 interactions to a 10 m target, 10,000 to a 23 m target, and 30,000 to the final target; maximum episode lengths are 70, 110, and 150. The selected policy weights transfer between stages while Adam state is reset; Stage 3 uses a $7.5e-5$ learning rate. This equalises total interactions but not final-task exposure, which is central to interpreting the result.

The curriculum targets are (95.190, -14.491, -3.0), (107.635, -10.842, -3.0), and (117.756, -19.034, -3.0), all from the fixed start (85.413, -15.334, -3.0). Each stage therefore extends the required displacement while retaining previously encountered geometry. Transfer is allowed only through model parameters; resetting Adam prevents stale momentum and second-moment estimates from dominating the changed task.

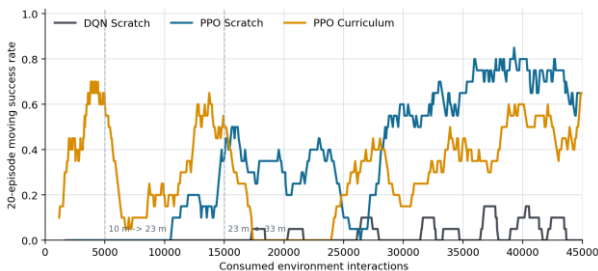


Fig. 1. Training success is highly variable. Curriculum stages before 15k interactions use easier targets; only Stage 3 matches the final route.

IV. EXPERIMENTAL SETUP

Experiments use AirSimNH with start (85.413, -15.334, -3.0) and final target (117.756, -19.034, -3.0). All three methods use Seed 7, 45,000 new environment interactions, the same observation/action/reward definitions, and a 150-step final-task limit. Measured training times were 7.41 h for DQN, 6.50 h for Scratch PPO, and 6.51 h for all curriculum stages combined.

TABLE I. CONTROLLED TRAINING CONFIGURATION

Method	Task exposure	Final LR	Selected
DQN-S	45k final route	$1e-4$	40.0k
PPO-S	45k final route	$1e-4$	42.5k
PPO-C	5k/10k/30k	$7.5e-5$	20.0k S3

Selected is the interaction count of the checkpoint chosen by the two-stage sweep; S3 denotes Curriculum Stage 3.

To reduce checkpoint-selection bias, every checkpoint is first evaluated for five episodes with the training seed; the top three are then evaluated for 30 episodes with Seed 10007. The selected model is tested once for 50 fresh episodes with Seed 20007. This seed changes stochastic simulator and policy sampling but not the route geometry. The primary deployment metric is deterministic success; secondary metrics are collision, timeout, altitude violation, reward, steps, path length, final distance, and minimum perceived depth. PPO is additionally evaluated stochastically to diagnose the learned action distribution.

Checkpoint ranking is lexicographic: higher success first, then lower unsafe rate, lower final distance, and fewer steps. Checkpoints are saved every 2,500 interactions, so selection can recover an earlier policy when continued optimisation degrades behaviour. The selected files are `dqn_step_0040000.pt`, `ppo_step_0042500.pt`, and the Stage 3 `ppo_step_0020000.pt`. The final Seed 20007 test is not reused for model choice.

A. Evaluation Measures

Success is the fraction of episodes ending within 2 m of the target. Collision and altitude rates quantify unsafe terminal events, while timeout identifies policies that remain safe but fail to finish. Mean steps and path length measure efficiency; mean final distance distinguishes near misses from severe failures; minimum front depth is a proxy for obstacle clearance. Return is reported because it reflects the optimised objective, but it is not used alone: shaped reward can increase even when operational success does not.

For PPO, deterministic evaluation uses $\arg\max$ over the six logits and approximates deployment with repeatable actions. Stochastic evaluation samples the categorical policy and probes whether non-maximal actions remain safe. The two modes use the same weights and environment; any gap is therefore evidence about probability mass in the actor, not additional training or a different checkpoint.

B. Reproducibility and Integrity Checks

Each run is isolated under `experiments/airsimnh/<algorithm>/<run>`, with separate models and results directories. The launcher starts the requested AirSim executable, waits for the RPC port, performs a clean-spawn smoke test, runs training or evaluation, records wall-clock time, and closes the scene on completion. At reset, the environment teleports the multirotor to the configured NED start, zeros motion, and records the collision timestamp. A collision flag inherited from scene spawn geometry is ignored only as a stale baseline; any newer timestamp terminates the episode. This avoids rewarding an agent for an invalid initial state while preserving subsequent collision detection.

Training logs store episode, cumulative interaction count, return, length, success, collision, altitude violation, final distance, exploration statistics, and PPO optimisation diagnostics. Evaluation adds complete per-step trajectories containing before/after position, depth clearance, action name and probability, every reward component, cumulative return, and terminal cause. Model files contain network parameters and step metadata, while the selected checkpoint is copied to a clearly named deterministic-best file.

These outputs allow the tables and plots in this report to be regenerated from CSV records rather than transcribed manually.

V. RESULTS

A. Training Behaviour

Episode outcomes during training are noisy because DQN follows epsilon-greedy exploration and PPO samples actions with a non-zero entropy bonus. Figure 1 should therefore be read as learning dynamics rather than final deployment performance. DQN does not establish a sustained high-success region. Scratch PPO develops successful final-route behaviour late in training, while curriculum success rises on the shorter stages and changes again when the target moves. This target-dependent discontinuity shows why Stage 1 and Stage 2 episodes cannot be counted as final-route successes.

TABLE II. INDEPENDENT FINAL TEST (DETERMINISTIC, N=50)

Method	Succ.	Coll.	TO	Reward	Dist.
DQN-S	16%	72%	14%	-18.5	4.47
PPO-S	98%	2%	0%	141.1	2.01
PPO-C	68%	2%	30%	71.2	14.66

Succ./Coll./TO are success, collision, and timeout rates; Dist. is mean final distance in metres.

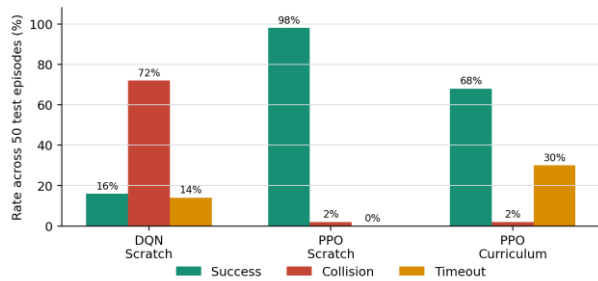


Fig. 2. Independent deterministic outcomes. Rates are reported separately because a terminal event at the step limit can overlap timeout bookkeeping.

Scratch PPO succeeds in 49/50 episodes (98%; Wilson 95% CI 89.5-99.6), versus 8/50 for DQN (16%; 8.3-28.5) and 34/50 for curriculum PPO (68%; 54.2-79.2). It also obtains the highest reward (141.1), shortest mean completion time (52.28 steps), and a 1.92 m mean minimum depth. Its 38.27 m path is 17.6% longer than the straight-line distance, consistent with a deliberate lateral detour rather than direct pursuit.

DQN collides in 72% of episodes, reaches a minimum perceived depth of only 0.37 m on average, and often alternates direct motion with hover near the obstacle. Curriculum PPO has the same low 2% deterministic collision rate as Scratch PPO, but times out in 30% of episodes and therefore ends 14.66 m from the target on average. Its 55.17 m average path indicates inefficient wandering in failed episodes.

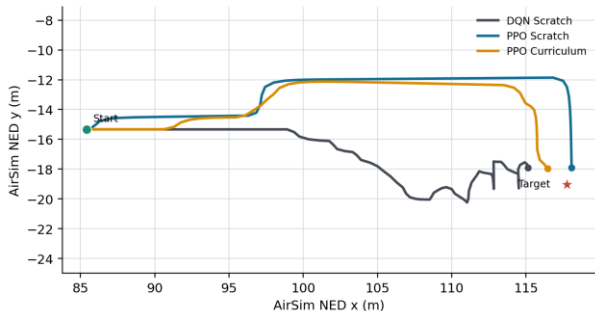


Fig. 3. Representative deterministic Episode 1 trajectories. Both PPO policies move laterally before approaching the target; DQN follows a lower-clearance route and collides.

The deployment mode matters. Scratch PPO falls from 98% deterministic success to 66% when actions are sampled; curriculum PPO falls from 68% to 46% and its collision rate rises from 2% to 38%. Thus the argmax policy is repeatable, but probability mass remains on unsafe alternatives. The reported mean final distance of 2.01 m for Scratch PPO is compatible with a 2 m success radius

because it averages 49 near-boundary successes with one collision failure.

TABLE III. PPO POLICY-MODE DIAGNOSTIC (N=50)

Method	Mode	Succ.	Coll.	TO	Reward
PPO-S	Det.	98%	2%	0%	141.1
PPO-S	Stoch.	66%	34%	0%	77.7
PPO-C	Det.	68%	2%	30%	71.2
PPO-C	Stoch.	46%	38%	16%	41.5

Det. selects the highest-probability action; Stoch. samples from the learned categorical distribution.

B. Failure Analysis

Per-step trajectories clarify the aggregate rates. DQN's average 0.37 m minimum depth and 72% collision rate indicate that its value estimates do not consistently trigger lateral avoidance before clearance becomes critical. Scratch PPO produces a repeatable side movement before returning toward the target, explaining both its 38.27 m path and high success. Its only deterministic failure keeps the average final distance slightly above the success radius despite 49 successful episodes.

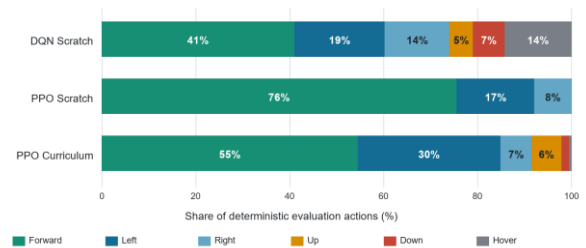


Fig. 4. Aggregate deterministic action distribution over all 50 final-test episodes. Percentages are computed from complete per-step trajectory logs.

Action usage supports the trajectory interpretation. Scratch PPO allocates 75.6% of its 2,614 actions to forward motion, 16.5% to left, and 7.8% to right, with no vertical or hover actions; it has learned a compact horizontal detour at the fixed target altitude. Curriculum PPO uses only 54.5% forward and 30.4% left, plus 8.0% vertical actions and 0.4% hover, consistent with its longer paths and timeouts. DQN is the least decisive: only 41.1% forward, with 14.2% hover and 25.6% combined vertical actions. These descriptive counts do not prove causality, but they identify the behavioural differences behind the endpoint metrics.

Curriculum PPO is safer than DQN but less decisive than Scratch PPO. Deterministic failures are predominantly timeouts rather than collisions, and the 14.66 m final distance shows that they are not merely one-step near misses. When sampled, both PPO policies expose substantially more collisions, demonstrating that evaluation of only the most likely action can conceal risky alternatives. For an autonomous system, deterministic performance is the correct deployment headline, but stochastic diagnostics remain important evidence about robustness.

C. Efficiency and Computational Cost

At 2 m/s for 0.35 s, an ideal forward command covers about 0.7 m. The 32.55 m straight-line displacement therefore has a nominal lower bound near 47 commands before accounting for acceleration, sensing, and avoidance. Scratch PPO averages 52.28 steps on the test, only about six commands above this idealised bound, while following a 38.27 m path. DQN and Curriculum PPO average 86.80 and 83.46 steps respectively; because their averages include collisions and timeouts, path length and step count must be interpreted jointly with success.

The shared encoder has 71,840 trainable convolutional parameters. Including fusion and output layers, DQN has approximately 1.684 million parameters and PPO 1.686 million, the small difference coming from LayerNorm and separate actor/critic heads. Thus PPO's advantage is not explained by a materially larger network. Measured throughput is about 1.69 interactions/s for DQN and 1.92 interactions/s for both PPO regimes, including AirSim control and

image transfer. Wall-clock differences are modest relative to the accuracy gap.

Checkpoint selection also improves computational efficiency at deployment: the best policies occur at 40.0k, 42.5k, and 20.0k Stage 3 interactions rather than necessarily at the final saved model. Continuing training after a useful policy can alter the actor or Q-function, so retaining intermediate checkpoints is essential in non-monotonic RL. The sweep does not create new capability; it identifies which already-trained policy should be evaluated and deployed.

VI. DISCUSSION AND CONCLUSION

The controlled comparison supports three conclusions. First, PPO is substantially better matched to this visual navigation task than the implemented vanilla DQN: clipped on-policy updates, GAE, and actor-critic learning produce a stable detour instead of repeatedly bootstrapping values from collision-heavy experience. Second, curriculum learning is not automatically beneficial. Spending one third of the budget on shorter goals reduces final-route exposure, and changing targets creates a distribution shift; the transferred policy is safe when successful but preserves short-horizon behaviour that manifests as timeouts. Third, deterministic and stochastic evaluation answer different questions: argmax measures deployment performance, while sampling reveals residual uncertainty.

The comparison also illustrates the difference between equal interaction cost and equal task exposure. All methods consume 45,000 transitions, but Scratch PPO receives 50% more final-route experience than Curriculum PPO (45,000 versus 30,000). The curriculum result is therefore not evidence that curricula are generally harmful; it shows that this particular distance schedule and fixed budget do not compensate for reduced Stage 3 rehearsal. A stronger curriculum would interleave earlier and final targets or allocate additional final-route steps while reporting both total and final-task budgets.

DQN remains a useful baseline despite its low score because it is a recognised image-based discrete-control method [2] and shares the same encoder and action interface. Its result establishes that the environment is not solved by arbitrary CNN value learning under this budget. Nevertheless, the comparison should not be interpreted as PPO outperforming every DQN variant: Double Q-learning [7], duelling heads, prioritised replay, longer exploration, or frame stacking could improve value estimation. The correct claim is that the implemented stable PPO configuration outperforms the documented vanilla DQN baseline on this controlled route.

A. What the Policy Has Learned

The simulator does not provide obstacle coordinates to the agent and the implementation does not build a map. Avoidance decisions are produced from the current inverse-depth pattern, relative target vector, velocity, and learned network weights. In this sense, the policy learns a visual action rule rather than recording a list of tree or building positions. Collision information affects gradients through reward and termination during training but is unavailable as an advance warning at inference.

However, a fixed start and goal make relative goal position strongly correlated with location along the route. A sufficiently expressive network can therefore associate particular goal vectors with left or right actions, effectively memorising a route phase even while receiving depth. The repeatable Scratch PPO detour and concentrated three-action distribution are compatible with both reactive avoidance and route memorisation. Only counterfactual tests, such as moving an obstacle or perturbing the start while keeping similar depth patterns, can separate these explanations. This is the main reason the present result is described as fixed-route navigation.

B. Reward Design and Safety

No direct reward is given for selecting a lateral action or maintaining a particular depth clearance. The agent must discover avoidance because a collision produces -100 and removes the future +100 goal opportunity. Dense progress feedback makes learning feasible, but it can also penalise the first part of a detour when distance

temporarily increases. The successful Scratch PPO trajectory shows that its long-term advantage estimates overcome this local conflict; DQN's bootstrap targets do so less consistently.

The 2 m success radius is deliberately larger than one nominal 0.7 m horizontal command, reducing oscillation around an exact coordinate. Altitude penalties are symmetric in terminal cost but NED signs remain asymmetric in implementation. The absence of altitude violations in all final tests suggests that the hold and margin terms work on this route, not that vertical safety is solved generally. A deployment-oriented extension should add side sensing, emergency braking, and a rule-based safety layer rather than relying only on learned penalties.

Strengths of the system include map-free visual control, a fair interaction budget, identical final-task evaluation, explicit safety metrics, checkpoint selection separated from testing, and action/reward trajectory logs that make failure analysis reproducible. The main weakness is external validity. There is only one training seed, one fixed route, one scene, one front depth frame, and no recurrent memory, yaw, backward action, domain randomisation, or real-drone validation. The test seed is independent of checkpoint selection but does not create unseen geometry, so these results must not be described as generalisation.

Internal validity is stronger than external validity. The methods share task definitions and final evaluation, and the 50-episode test gives useful uncertainty intervals: Scratch PPO's 98% success has a Wilson 95% interval of 89.5-99.6%, compared with 54.2-79.2% for Curriculum PPO and 8.3-28.5% for DQN. However, these intervals measure episode variability for the selected trained policies; they do not account for variation across training seeds. One unusually good or poor Seed 7 optimisation run could therefore change the algorithm ranking.

Future work should repeat training across at least three to five seeds, perturb safe start and target positions, evaluate a second AirSim environment, and train on multiple routes with weather and texture randomisation. Algorithmic extensions should compare Double/Dueling DQN, recurrent visual memory, continuous-control PPO or SAC, and a curriculum that preserves final-task rehearsal. Within the current scope, deterministic Scratch PPO is the recommended policy: it achieves 98% success and 2% collision on the held-out 50-episode fixed-route test.

A practical next experiment is a route-generalisation matrix: train each method on the current route using three seeds, then evaluate deterministic policies on several safe start/goal perturbations and one unseen scene without further learning. Reporting mean and standard deviation across seeds, success versus perturbation distance, and collision objects would separate memorisation of a fixed detour from transferable visual obstacle avoidance. Until that experiment is performed, the defensible conclusion is high fixed-route deployment performance for Scratch PPO, not general autonomous navigation.

REFERENCES

- [1] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles," in *Field and Service Robotics*, 2018, pp. 621-635.
- [2] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529-533, 2015.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.
- [4] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," in *Proc. ICML*, 2009, pp. 41-48.
- [5] Microsoft Research, "AirSim," GitHub repository. Available: <https://github.com/microsoft/AirSim>.
- [6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [7] H. van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-learning," in *Proc. AAAI*, 2016, pp. 2094-2100.